

英语学习Agent系列

打造你的专属英语学习助手

AI技术专栏

2026年

目录

1. 为什么需要英语学习Agent
2. 词汇学习模块
3. 语法学习模块
4. 阅读理解模块
5. 口语练习模块
6. 写作辅助模块
7. Web界面集成
8. 项目部署与总结

1. 为什么需要英语学习Agent

为什么需要英语学习Agent

系列导航：

【本地大模型知识库系列】 01-12篇（已完成） |

【英语学习Agent系列】 当前第01篇（共8篇）

为什么需要英语学习Agent

AI驱动的个性化语言学习

英语学习Agent系列第01篇 | 架构设计

在前面的【本地大模型知识库系列】中，我们学习了如何搭建一个基于RAG的本地知识库系统。现在，让我们将这个基础升级为一个更有价值的应用——英语学习Agent。

01

传统英语学习的痛点

传统的英语学习方式存在很多问题：

传统学习的痛点

-

缺乏个性化：

老师面对几十个学生，无法根据每个人的水平和学习习惯制定专属计划

-

反馈不及时：

作业批改需要时间，学生无法立即知道自己的错误

-

练习场景有限：

口语练习机会少，缺乏真实对话场景

-

复习效率低：

没有科学的记忆曲线安排，容易遗忘

-

学习路径模糊：

不知道该学什么、学到什么程度

02

AI学习Agent的优势

一个智能的英语学习Agent可以解决这些问题：

AI学习Agent的优势

-

个性化学习：

根据用户水平和学习习惯自动调整难度和内容

-

即时反馈：

作文、口语练习可以立即得到批改和建议

-

无限练习场景：

可以模拟各种真实对话场景进行口语练习

-

科学复习：

基于记忆曲线算法安排复习计划

-

清晰路径：

可视化的学习路径和进度追踪

03

英语学习Agent架构

我们的英语学习Agent基于之前搭建的本地知识库系统，架构如下：

整个系统分为五层：

用户界面层：

使用Gradio构建友好的学习界面（参考【本地大模型知识库系列】第04/09篇）

Agent编排层：

使用LangChain Agent控制学习流程和对话逻辑

知识库层：

使用Chroma存储英语学习资料（参考【本地大模型知识库系列】第03篇）

大模型层：

使用Ollama运行本地大模型进行语言理解和生成（参考【本地大模型知识库系列】第02篇）

知识图谱层：

使用NetworkX构建知识点依赖关系，支持学习路径规划

04

核心功能模块

系统包含六个核心学习模块：

模块

功能描述

技术实现

词汇学习

单词释义、例句、记忆曲线复习、听写练习

Spaced Repetition + Prompt工程

语法学习

语法点讲解、例句分析、填空练习

知识图谱 + 练习生成

阅读理解

文章阅读、问题回答、难度自适应

RAG检索（参考第07篇）

写作练习

主题写作、自动批改、润色建议

LLM生成 + 评分标准

听力训练

音频播放、听写练习、听力理解

TTS + 语音识别

口语练习

情景对话、发音纠正、话题讨论

STT + LLM对话

常见问题

1. 需要什么样的硬件配置？：建议至少16GB内存，有GPU更好（参考【本地大模型知识库系列】第02篇）
2. 数据安全吗？：所有数据存储在本地，无需上传到云端（参考【本地大模型知识库系列】第01篇）
3. 支持哪些英语水平？：从入门到高级，系统会自动检测用户水平
4. 需要联网吗？：基础功能无需联网，语音合成需要网络（可选）

—— 未完待续 ——

系列导航：

- 本地大模型知识库系列（12篇，已完成）
- 英语学习Agent系列（8篇，当前第01篇）

英语学习Agent系列第01篇

下一篇：词汇学习模块——构建智能单词本

2. 词汇学习模块

词汇学习模块——构建智能单词本

系列导航：

【本地大模型知识库系列】 01-12篇（已完成） |

【英语学习Agent系列】 当前第02篇（共8篇）

词汇学习模块

构建智能单词本

英语学习Agent系列第02篇 | 词汇学习

词汇是英语学习的基础。一个好的单词本不仅要展示单词的释义，还要根据记忆曲线算法合理安排复习时间，让学习事半功倍。

01

记忆曲线算法

艾宾浩斯记忆曲线告诉我们，人的记忆会随着时间逐渐遗忘。通过在遗忘临界点进行复习，可以有效强化记忆。我们实现一个简化的间隔重复算法：

python

import

datetime

from

enum

import

Enum

class

ReviewInterval

(

Enum

):

NEW

0

ONE_DAY

1

THREE_DAYS

3

ONE_WEEK

7

TWO_WEEKS

14

ONE_MONTH

30

THREE_MONTHS

90

class

WordCard

:

def

__init__

(

self

,

word

,

definition

,

example

""

):

self

.

word

word

self

.

definition

definition

self

.

example

example

self

.

level

ReviewInterval

.

NEW

self

.

last_review

None

self

.

next_review

None

self

.

correct_count

0

self

.

wrong_count

0

def

review

(

self

,

is_correct

```
):  
  
self  
  
.   
  
last_review  
  
  
datetime  
  
.   
  
datetime  
  
.   
  
now  
  
()  
  
if  
  
is_correct  
  
:  
  
self  
  
.   
  
correct_count  
  
+=  
  
1  
  
self  
  
.   
  
_level_up  
  
()  
  
else  
  
:  
  
self  
  
.   

```

wrong_count

+=

1

self

.

_level_down

()

self

.

_update_next_review

()

def

_level_up

(

self

):

levels

[

ReviewInterval

.

NEW

,

ReviewInterval

.

ONE_DAY

,

ReviewInterval

.

THREE_DAYS

,

ReviewInterval

.

ONE_WEEK

,

ReviewInterval

.

TWO_WEEKS

,

ReviewInterval

.

ONE_MONTH

,

ReviewInterval

.

THREE_MONTHS

]

current_idx

levels

.

index

(

self

```
.  
level  
)  
self  
.  
level  
  
levels  
[  
min  
(  
current_idx  
+  
1  
,  
len  
(  
levels  
)  
  
1  
)  
]  
def  
_level_down  
(  
self
```

```
):  
  
self  
  
.  
  
level  
  
  
ReviewInterval  
  
.  
  
NEW  
  
def  
  
_update_next_review  
  
(  
  
self  
  
):  
  
self  
  
.  
  
next_review  
  
  
datetime  
  
.  
  
datetime  
  
.  
  
now  
  
( ) \  
  
+  
  
datetime  
  
.  
  
timedelta
```

```
(  
    days  
  
    self  
    .  
    level  
    .  
    value  
)  
  
def  
is_due_for_review  
(  
    self  
)  
:  
    if  
        self  
        .  
        next_review  
        is  
        None  
    :  
        return  
        True  
    return  
        datetime  
        .  
        datetime
```

.

now

()

>=

self

.

next_review

02

使用Ollama生成单词卡

利用大模型生成单词的详细解释和例句（关于Ollama的安装和使用，请参考【本地大模型知识库系列】第02篇）：

python

import

ollama

import

json

class

WordCardGenerator

:

def

__init__

(

self

,

model

"llama3:8b"

```
):
self
.
model

model

def
generate_word_card
(
self
,
word
):
prompt

f"""
```

你是一个英语词汇专家。请为单词 "{word}" 生成详细的学习卡片，包括：

- 1. 音标（英音和美音）
- 2. 词性
- 3. 中文释义（多个含义用分号分隔）
- 4. 常用搭配（至少3个）
- 5. 例句（至少2个，中英文对照）
- 6. 记忆技巧

请以JSON格式输出，不要包含其他内容。格式如下：

```
{{
"word": "单词",
"phonetic": "音标",
```

```
"part_of_speech": "词性",

"definitions": ["释义1", "释义2"],

"collocations": ["搭配1", "搭配2", "搭配3"],

"examples": [{"english": "英文例句", "chinese": "中文翻译"}],

"memory_tip": "记忆技巧"

}}
```

"""

response

ollama

.

chat

(

model

self

.

model

,

messages

"role"

:

"user"

,

"content"

:

prompt

```
    }  
  )  
  try  
  :  
  data  
  
  json  
  .  
  loads  
  (  
  response  
  [  
  "message"  
  ]  
  "content"  
  ]  
  )  
  return  
  data  
  except  
  json  
  .  
  JSONDecodeError  
  :  
  return  
  None  
  
# 使用示例
```

generator

WordCardGenerator

()

card

generator

.

generate_word_card

(

"ubiquitous"

)

if

card

:

print

(

json

.

dumps

(

card

,

ensure_ascii

False

,

indent

2

)

)

03

单词本管理

创建一个单词本管理器，支持添加单词、获取待复习单词、记录复习结果：

```
python
```

```
import
```

```
json
```

```
import
```

```
os
```

```
class
```

```
WordBook
```

```
:
```

```
def
```

```
__init__
```

```
(
```

```
self
```

```
,
```

```
file_path
```

```
"word_book.json"
```

```
):
```

```
self
```

```
.
```

```
file_path
```

file_path

self

.

cards

{}

self

.

_load

()

def

_load

(

self

):

if

os

.

path

.

exists

(

self

.

file_path

):

with

```
open
(
self
.
file_path
,
'r'
,
encoding

'utf-8'
)
as
f
:
data

json
.
load
(
f
)
for
word
,
card_data
```

```
in
data
.
items
():
card

WordCard
(
word
,
card_data
[
"definition"
],
card_data
[
"example"
])
card
.
level

ReviewInterval
[
card_data
[
```

"level"

]

]

card

.

correct_count

card_data

[

"correct_count"

]

card

.

wrong_count

card_data

[

"wrong_count"

]

self

.

cards

[

word

] =

card

def

```
_save  
  
(  
    self  
):  
    data  
  
    {  
        for  
            word  
        ,  
            card  
        in  
            self  
        .  
            cards  
        .  
            items  
    }():  
        data  
        [  
            word  
        ] =  
        {  
            "definition"  
        }:  
            card  
        .
```

definition

,

"example"

:

card

.

example

,

"level"

:

card

.

level

.

name

,

"correct_count"

:

card

.

correct_count

,

"wrong_count"

:

card

.

```
wrong_count
```

```
}
```

```
with
```

```
open
```

```
(
```

```
self
```

```
.
```

```
file_path
```

```
,
```

```
'w'
```

```
,
```

```
encoding
```

```
'utf-8'
```

```
)
```

```
as
```

```
f
```

```
:
```

```
json
```

```
.
```

```
dump
```

```
(
```

```
data
```

```
,
```

```
f
```

```
,
```

```
ensure_ascii
```

False

,

indent

2

)

def

add_word

(

self

,

word

,

definition

,

example

"""

):

if

word

not

in

self

.

cards

:

```
self
.
cards
[
word
] =
WordCard
(
word
,
definition
,
example
)
self
.
_save
()
def
get_due_cards
(
self
,
limit
10
):
```

due_cards

[

card

for

card

in

self

.

cards

.

values

()

if

card

.

is_due_for_review

()

]

return

due_cards

[:

limit

]

def

record_review

(

```
self
,
word
,
is_correct
):
if
word
in
self
.
cards
:
self
.
cards
[
word
]
.
review
(
is_correct
)
self
.
```

_save

()

04

复习流程

python

class

VocabularyReviewer

:

def

__init__

(

self

):

self

.

word_book

WordBook

()

self

.

generator

WordCardGenerator

()

def

start_review

```
(  
    self  
    ,  
    count  
  
    10  
):  
    due_cards  
  
    self  
    .  
    word_book  
    .  
    get_due_cards  
    (  
        count  
    )  
    if  
        not  
            due_cards  
        :  
            return  
            {  
                "message"  
            }  
            :  
            "没有需要复习的单词！"  
    }
```

return

{

"message"

:

f"今日需要复习{len(due_cards)}个单词"

,

"cards"

:

[

{

"word"

:

card

.

word

,

"definition"

:

card

.

definition

,

"example"

:

card

.

example

}

for

card

in

due_cards

]

}

def

add_new_word

(

self

,

word

):

card_data

self

.

generator

.

generate_word_card

(

word

)

if

card_data

:

definition

"; ".join

(

card_data

[

"definitions"

]

)

example

"\n".join

(

[

f

"{e['english']} – {e['chinese']}"

for

e

in

card_data

[

"examples"

]

]

)

self

```
.
word_book

.
add_word
(
word
,
definition
,
example
)
return
card_data
else
:
return
{
"error"
:
"生成单词卡失败"
}
```

常见问题

1. 生成的单词卡格式错误：检查Ollama是否正常运行（参考【本地大模型知识库系列】第02篇）
2. 复习间隔不合理：调整ReviewInterval的时间间隔
3. 单词本数据丢失：确保文件保存路径有写入权限
4. 生成速度慢：使用更小的模型或添加缓存机制（参考【本地大模型知识库系列】第10篇）

—— 未完待续 ——

系列导航：

- 本地大模型知识库系列（12篇，已完成）
- 英语学习Agent系列（8篇，当前第02篇）

英语学习Agent系列第02篇

下一篇：语法学习模块——知识图谱驱动的语法教学

3. 语法学习模块

语法学习模块——知识图谱驱动的语法教学

系列导航：

【本地大模型知识库系列】 01-12篇（已完成） |

【英语学习Agent系列】 当前第03篇（共8篇）

语法学习模块

知识图谱驱动的语法教学

英语学习Agent系列第03篇 | 语法学习

语法是英语学习的骨架。传统语法学习往往是零散的知识点记忆，缺乏系统性。我们将使用知识图谱来构建语法知识点之间的依赖关系，让学习更有逻辑性。

01

构建语法知识图谱

使用NetworkX构建语法知识点的依赖关系图谱：

```
python
import
networkx
as
nx
import
json
class
GrammarKnowledgeGraph
:
def
```

```
__init__
(
self
):
self
.
graph

nx
.
DiGraph
()
self
.
_build_graph
()
def
_build_graph
(
self
):
grammar_topics

{
"基础句型"
:
{
```

"description"

:

"主谓宾、主系表等基本句型结构"

,

"level"

:

1

,

"prerequisites"

:

[]

},

"名词"

:

{

"description"

:

"可数名词、不可数名词、名词所有格"

,

"level"

:

1

,

"prerequisites"

:

[]

},

"冠词"

:

{

"description"

:

"a/an/the的用法和区别"

,

"level"

:

1

,

"prerequisites"

:

[

"名词"

]

},

"代词"

:

{

"description"

:

"人称代词、物主代词、反身代词"

,

"level"

:
1
,
"prerequisites"

:
[],
"一般现在时"

:
{
"description"

:
"经常性动作和客观事实"
,
"level"

:
2
,
"prerequisites"

:
[
"基础句型"

,
"动词"

],
},

"一般过去时"

:

{

"description"

:

"过去发生的动作"

,

"level"

:

2

,

"prerequisites"

:

[

"一般现在时"

]

},

"一般将来时"

:

{

"description"

:

"将要发生的动作"

,

"level"

:

2

,

"prerequisites"

:

[

"一般现在时"

]

},

"现在进行时"

:

{

"description"

:

"正在发生的动作"

,

"level"

:

2

,

"prerequisites"

:

[

"一般现在时"

]

},

"动词"

:

{

"description"

:

"及物动词、不及物动词、系动词"

,

"level"

:

1

,

"prerequisites"

:

[]

},

"情态动词"

:

{

"description"

:

"can/may/must/should等"

,

"level"

:

3

,

"prerequisites"

:
[
"动词"
,
"一般现在时"
]
},
"非谓语动词"

:
{
"description"

:
"不定式、动名词、分词"
,
"level"

:
4
,
"prerequisites"

:
[
"动词"
,
"一般现在时"
]
},

"定语从句"

:

{

"description"

:

"修饰名词的从句"

,

"level"

:

4

,

"prerequisites"

:

[

"名词"

,

"基础句型"

]

},

"状语从句"

:

{

"description"

:

"修饰动词/形容词/副词的从句"

,

"level"

:

4

,

"prerequisites"

:

[

"基础句型"

,

"一般现在时"

]

},

"虚拟语气"

:

{

"description"

:

"与事实相反的假设"

,

"level"

:

5

,

"prerequisites"

:

[

"情态动词"

,

"一般过去时"

]

}

}

for

topic

,

info

in

grammar_topics

.

items

():

self

.

graph

.

add_node

(

topic

, **info

)

for

prereq

```
in
info
[
"prerequisites"
]:
self
.
graph
.
add_edge
(
prereq
,
topic
,
relation

"prerequisite"
)
def
get_prerequisites
(
self
,
topic
):
if
```

```
topic
not
in
self
.
graph
.
nodes
:
return
[]
return
list
(
self
.
graph
.
predecessors
(
topic
))
def
get_next_topics
(
self
```

```
,
topic
):
if
topic
not
in
self
.
graph
.
nodes
:
return
[]
return
list
(
self
.
graph
.
successors
(
topic
))
```

```
def
get_learning_path
(
self
,
start_topic

):
if
start_topic
is
None
:
start_nodes

[
n
for
n
in
self
.
graph
.
nodes
if
self
```

```
.
graph
.
in_degree
(
n
)

0
]
start_topic

start_nodes
[
0
]
path

[]
visited

set
()
self
.
_dfs_path
(
start_topic
```

```
,  
visited  
  
,  
path  
)  
return  
path  
def  
_dfs_path  
(  
self  
,  
topic  
,  
visited  
,  
path  
):  
if  
topic  
in  
visited  
:  
return  
visited  
.
```

```
add
(
topic
)
path
.
append
(
topic
)
for
successor
in
sorted
(
self
.
get_next_topics
(
topic
)):
self
.
_dfs_path
(
successor
```

```
,  
visited  
  
,  
path  
)
```

02

语法点讲解与练习生成

使用Ollama生成语法点的详细讲解和练习题：

```
python  
  
import  
  
ollama  
  
import  
  
json  
  
class  
  
GrammarTutor  
  
:  
  
def  
  
__init__  
  
(  
  
self  
  
,  
  
model  
  
"llama3:8b"  
  
):  
  
self  
  
.
```

model

model

self

.

graph

GrammarKnowledgeGraph

()

def

explain_topic

(

self

,

topic

):

prerequisites

self

.

graph

.

get_prerequisites

(

topic

)

prompt

f"""

你是一位专业的英语语法老师。请详细讲解语法点"{topic}"：

要求：

- 1. 简明扼要的定义和核心规则
- 2. 常见用法和注意事项
- 3. 至少3个典型例句（中英文对照）
- 4. 常见错误和纠正方法
- 5. 如果有前置知识"{prerequisites}"，请简要回顾相关内容

请以JSON格式输出：

```
{{
  "topic": "语法点名称",
  "definition": "定义",
  "rules": ["规则1", "规则2", "规则3"],
  "examples": [{"english": "例句", "chinese": "翻译", "explanation": "解析"}],
  "common_mistakes": [{"wrong": "错误句子", "correct": "正确句子", "reason": "原因"}],
  "difficulty": "难度等级(1-5)"
}}
```

"""

response

ollama

.

chat

(

model

self

```
.
model

,

messages

"role"

:

"user"

,

"content"

:

prompt
}]

)

try

:

return

json

.

loads

(

response

[

"message"

]]

"content"

]
```

```
)

except

json

.

JSONDecodeError

:

return

None

def

generate_exercises

(

    self

    ,

    topic

    ,

    count

    5

):

    prompt

    f"""

    你是一位专业的英语语法老师。请为语法点"{topic}"生成{count}道练习题。

    题型包括：

    1. 选择题

    2. 填空题

    3. 改错题
```

请以JSON格式输出：

```
{{
  "topic": "{topic}",
  "exercises": [
    {{
      "type": "choice/fill/error",
      "question": "题目内容",
      "options": ["选项A", "选项B", "选项C", "选项D"],
      "answer": "正确答案",
      "explanation": "答案解析"
    }}
  ]
}}
```

response

ollama

.

chat

(

model

self

.

model

,

messages

```
"role"  
  
:  
  
"user"  
  
,  
  
"content"  
  
:  
  
prompt  
  
}]  
  
)  
  
try  
  
:  
  
return  
  
json  
  
.  
  
loads  
  
(  
  
response  
  
[  
  
"message"  
  
][  
  
"content"  
  
]  
  
)  
  
except  
  
json  
  
.
```

JSONDecodeError

:

return

None

def

check_answer

(

self

,

exercise

,

user_answer

):

is_correct

user_answer

.

strip

()

.

lower

()

exercise

[

"answer"

]

.

strip

()

.

lower

()

return

{

"correct"

:

is_correct

,

"correct_answer"

:

exercise

[

"answer"

]

,

"explanation"

:

exercise

[

"explanation"

]

}

03

语法学习流程

python

class

GrammarLearner

:

def

__init__

(

self

):

self

.

tutor

GrammarTutor

()

self

.

completed_topics

set

()

self

.

current_topic

None

```
def
start_topic
(
self
,
topic
):
prerequisites

self
.
tutor
.
graph
.
get_prerequisites
(
topic
)
for
prereq
in
prerequisites
:
if
prereq
not
```

```
in
self
.
completed_topics
:
return
{
"status"
:
"error"
,
"message"
:
f"需要先学习前置知识：{prerequisites}"
}
self
.
current_topic

topic
explanation

self
.
tutor
.
explain_topic
```

```
(  
  topic  
)  
  
exercises  
  
self  
.  
tutor  
.  
generate_exercises  
(  
  topic  
)  
return  
{  
  "status"  
  :  
  "success"  
  ,  
  "topic"  
  :  
  topic  
  ,  
  "explanation"  
  :  
  explanation  
  ,
```

"exercises"

:

exercises

[

"exercises"

]

if

exercises

else

[]

}

def

submit_answer

(

self

,

exercise

,

answer

):

return

self

.

tutor

.

check_answer

```
(  
    exercise  
    ,  
    answer  
)  
  
def  
    complete_topic  
    (  
        self  
        ,  
        topic  
    ):  
    self  
    .  
    completed_topics  
    .  
    add  
    (  
        topic  
    )  
  
    next_topics  
  
    self  
    .  
    tutor  
    .  
    graph
```

```
.  
  
get_next_topics  
  
(  
  
topic  
  
)  
  
return  
  
{  
  
"message"  
  
:  
  
f"恭喜完成'{topic}'！"  
  
,  
  
"next_topics"  
  
:  
  
next_topics  
  
,  
  
"completed_count"  
  
:  
  
len  
  
(  
  
self  
  
.completed_topics  
  
)  
  
}  
  
def  
  
get_learning_progress
```

```
(
    self
):
    total_topics

    len

    (
        self
        .
        tutor
        .
        graph
        .
        graph
        .
        nodes
    )

    completed_count

    len

    (
        self
        .
        completed_topics
    )

    return

{
```

```
"completed_topics"
```

```
:
```

```
list
```

```
(
```

```
self
```

```
.
```

```
completed_topics
```

```
),
```

```
"total_topics"
```

```
:
```

```
total_topics
```

```
,
```

```
"progress"
```

```
:
```

```
round
```

```
(
```

```
completed_count
```

```
/
```

```
total_topics
```

```
*
```

```
100
```

```
,
```

```
2
```

```
)
```

```
}
```

常见问题

1. 知识点依赖关系不正确：可以手动调整知识图谱中的prerequisites配置
2. 练习题难度不合适：在generate_exercises中添加难度参数
3. 进度数据丢失：添加进度持久化存储（参考第02篇的WordBook实现）
4. 讲解内容太复：优化prompt，要求更简洁易懂的解释

—— 未完待续 ——

系列导航：

- 本地大模型知识库系列（12篇，已完成）
- 英语学习Agent系列（8篇，当前第03篇）

英语学习Agent系列第03篇

下一篇：阅读理解模块——自适应难度阅读

4. 阅读理解模块

阅读理解模块——自适应难度阅读

系列导航：

【本地大模型知识库系列】 01-12篇（已完成） |

【英语学习Agent系列】 当前第04篇（共8篇）

阅读理解模块

自适应难度阅读

英语学习Agent系列第04篇 | 阅读理解

阅读理解是英语学习的核心能力之一。传统阅读材料往往难度固定，无法根据学习者水平动态调整。我们将构建一个自适应难度的阅读理解系统，根据学习者的表现自动调整文章难度，并生成针对性的练习题目。

01

文章难度评估与分级

使用Flesch–Kincaid可读性指数和词汇难度来评估文章难度：

python

import

textstat

import

re

class

DifficultyEvaluator

:

def

__init__

(

self

):

self

.

cefr_levels

{

1

:

"A1"

,

2

:

"A2"

,

3

:

"B1"

,

4

:

"B2"

,

5

:

"C1"

,

6

:

"C2"

}

def

evaluate

(

self

,

text

):

fk_grade

textstat

.

flesch_kincaid_grade

(

text

)

flesch_ease

textstat

.

flesch_reading_ease

(

text

)

sentence_count

textstat

.

sentence_count

(

text

)

word_count

textstat

.

lexicon_count

(

text

,

removepunct

True

)

avg_sentence_length

word_count

/

sentence_count

if

sentence_count

>

0

else

0

cefr_level

self

.

_map_to_cefr

(

fk_grade

)

return

{

"fk_grade"

:

round

(

fk_grade

,

2

),

"flesch_ease"

:

round

(

flesch_ease

,

2

),

"avg_sentence_length"

:

round

(

avg_sentence_length

,

2

),

"word_count"

:

word_count

,

"sentence_count"

:

sentence_count

,

"cefr_level"

:

cefr_level

,

"difficulty_score"

:

self

.

```
_calculate_difficulty_score
```

```
(
```

```
fk_grade
```

```
,
```

```
avg_sentence_length
```

```
)
```

```
}
```

```
def
```

```
_map_to_cefr
```

```
(
```

```
self
```

```
,
```

```
fk_grade
```

```
):
```

```
if
```

```
fk_grade
```

```
<=
```

```
3
```

```
:
```

```
return
```

```
"A1"
```

```
elif
```

```
fk_grade
```

```
<=
```

```
5
```

```
:
```

return

"A2"

elif

fk_grade

<=

7

:

return

"B1"

elif

fk_grade

<=

9

:

return

"B2"

elif

fk_grade

<=

11

:

return

"C1"

else

:

return

"C2"

def

_calculate_difficulty_score

(

self

,

fk_grade

,

avg_sentence_length

):

return

round

(

fk_grade

*

0.6

+

avg_sentence_length

*

0.4

,

2

)

02

文章库管理与检索

使用向量数据库存储文章，支持按难度检索（关于向量数据库的使用，请参考【本地大模型知识库系列】第03篇）：

```
python
```

```
import
```

```
chromadb
```

```
import
```

```
json
```

```
class
```

```
ArticleLibrary
```

```
:
```

```
def
```

```
__init__
```

```
(
```

```
self
```

```
,
```

```
persist_directory
```

```
"/chroma_db"
```

```
):
```

```
self
```

```
.
```

```
client
```

```
chromadb
```

```
.
```

```
PersistentClient
```

```
(
```

```
path
```

```
persist_directory
```

```
)

self
.

collection

self
.

client
.

get_or_create_collection

(

"english_articles"

)

self
.

evaluator

DifficultyEvaluator

()

def
add_article

(

self
,

title
,

content
```

```
,
category

"general"

):

difficulty

self

.

evaluator

.

evaluate

(

content

)

metadata

{

"title"

:

title

,

"category"

:

category

,

"cefr_level"

:
```

difficulty

[

"cefr_level"

]

,

"difficulty_score"

:

difficulty

[

"difficulty_score"

]

,

"word_count"

:

difficulty

[

"word_count"

]

}

self

.

collection

.

add

(

documents

content

],

metadatas

metadata

],

ids

title

]

)

return

difficulty

def

get_articles_by_difficulty

(

self

,

cefr_level

,

limit

5

):

results

self

.

collection

.

query

(

query_texts

" "

],

where

"cefr_level"

:

cefr_level

},

n_results

limit

)

return

[{

"title"

:

meta

[

"title"

]

,

"content"

```
:  
  
doc  
  
,  
  
"cefr_level"  
  
:  
  
meta  
  
[  
  
"cefr_level"  
  
]  
  
,  
  
"difficulty_score"  
  
:  
  
meta  
  
[  
  
"difficulty_score"  
  
]  
  
}  
  
for  
  
doc  
  
,  
  
meta  
  
in  
  
zip  
  
(  
  
results  
  
[
```

"documents"

][

0

],

results

[

"metadatas"

][

0

]

)

]

def

search_articles

(

self

,

query

,

cefr_level

None

,

limit

5

):

where_clause

{

"cefr_level"

:

cefr_level

}

if

cefr_level

else

None

results

self

.

collection

.

query

(

query_texts

query

],

where

where_clause

,

n_results

```
limit
)
return
[
  {
    "title"
    :
    meta
    [
      "title"
    ]
    ,
    "content"
    :
    doc
    ,
    "cefr_level"
    :
    meta
    [
      "cefr_level"
    ]
    ,
    "difficulty_score"
    :
    meta
    [
```

"difficulty_score"

]

}

for

doc

,

meta

in

zip

(

results

[

"documents"

][

0

],

results

[

"metadatas"

][

0

]

)

]

03

阅读理解练习生成

使用Ollama根据文章内容生成阅读理解题目：

python

import

ollama

import

json

class

ReadingComprehensionGenerator

:

def

__init__

(

self

,

model

"llama3:8b"

):

self

.

model

model

def

generate_questions

(

self

，
article_title

，
article_content

，
question_count

5

):

prompt

f"""

你是一位专业的英语老师。请根据以下文章生成{question_count}道阅读理解题目。

文章标题：{article_title}

文章内容：

{article_content}

题目类型包括：

- 1. 主旨大意题 (main idea)
- 2. 细节理解题 (detail)
- 3. 推理判断题 (inference)
- 4. 词义猜测题 (vocabulary)

请以JSON格式输出，不要包含其他内容：

{{

"title": "文章标题",

"questions": [

{{

"type": "main idea/detail/inference/vocabulary",

"question": "题目内容",
"options": ["选项A", "选项B", "选项C", "选项D"],
"answer": "正确答案（A/B/C/D）",
"explanation": "答案解析"

}}

]

}}

""

response

ollama

.

chat

(

model

self

.

model

,

messages

"role"

:

"user"

,

"content"

:

prompt

}]

)

try

:

return

json

.

loads

(

response

[

"message"

][

"content"

]

)

except

json

.

JSONDecodeError

:

return

None

def

generate_summary

```
(
self
,
article_content
):
prompt

f"""
```

请为以下英语文章生成中文摘要（100–150字），并列5个重点单词：

文章内容：

```
{article_content}
```

请以JSON格式输出：

```
{{
"summary": "中文摘要",
"key_words": ["单词1", "单词2", "单词3", "单词4", "单词5"]
}}
```

response

ollama

.

chat

(

model

self

.

model

```
,
messages

"role"

:

"user"

,

"content"

:

prompt

}]

)

try

:

return

json

.

loads

(

response

[

"message"

][

"content"

]

)

except
```

```
json
```

```
.
```

```
JSONDecodeError
```

```
:
```

```
return
```

```
None
```

```
04
```

自适应难度调整

根据学习者表现动态调整文章难度：

```
python
```

```
import
```

```
json
```

```
import
```

```
os
```

```
class
```

```
ReadingLevelManager
```

```
:
```

```
def
```

```
__init__
```

```
(
```

```
self
```

```
,
```

```
file_path
```

```
"reading_level.json"
```

```
):
```

```
self
```

.

file_path

file_path

self

.

level

"B1"

self

.

accuracy_history

[]

self

.

_load

()

self

.

cefr_order

[

"A1"

,

"A2"

,

"B1"

```
,  
"B2"  
  
,  
"C1"  
  
,  
"C2"  
]  
  
def  
_load  
(  
self  
):  
if  
os  
.  
path  
.  
exists  
(  
self  
.  
file_path  
):  
with  
open  
(
```

```
self
.
file_path
,
'r'
,
encoding

'utf-8'

)

as

f

:

data

json

.

load

(

f

)

self

.

level

data

.

get
```

```
(  
    "level"  
    ,  
    "B1"  
)  
  
self  
.  
accuracy_history  
  
data  
.  
get  
(  
    "accuracy_history"  
    ,  
    []  
)  
  
def  
    _save  
(  
    self  
    ):  
    data  
  
    {  
        "level"  
        :
```

```
self
.
level
,
"accuracy_history"
:
self
.
accuracy_history
}
with
open
(
self
.
file_path
,
'w'
,
encoding
'utf-8'
)
as
f
:
json
```

```
.  
  
dump  
  
(  
  
data  
  
,  
  
f  
  
,  
  
ensure_ascii  
  
False  
  
,  
  
indent  
  
2  
  
)  
  
def  
  
record_score  
  
(  
  
self  
  
,  
  
correct_count  
  
,  
  
total_count  
  
):  
  
accuracy  
  
correct_count  
  
/
```

total_count

if

total_count

>

0

else

0

self

.

accuracy_history

.

append

(

accuracy

)

if

len

(

self

.

accuracy_history

)

>

10

:

self

.

accuracy_history

self

.

accuracy_history

[

]

self

.

_adjust_level

()

self

.

_save

()

def

_adjust_level

(

self

):

if

len

(

self

.

accuracy_history

)

<

5

:

return

avg_accuracy

sum

(

self

.

accuracy_history

[

])

/

5

current_idx

self

.

cefr_order

.

index

(

self

.

```
level
)
if
avg_accuracy
>=
0.85
and
current_idx
<
len
(
self
.
cefr_order
)

1
:
self
.
level

self
.
cefr_order
[
current_idx
```

```
+
1
]
print
(
f" 升级到 {self.level} 级别！"
)
elif
avg_accuracy
<
0.5
and
current_idx
>
0
:
self
.
level

self
.
cefr_order
[
current_idx
1
```

```
]

print

(

f" 降级到 {self.level} 级别"

)

def

get_current_level

(

self

):

return

{

"level"

:

self

.

level

,

"recent_accuracy"

:

round

(

sum

(

self

.


```

accuracy_history

[

])

/

max

(

len

(

self

.

accuracy_history

[

]),

1

),

2

)

}

05

阅读学习流程

python

class

ReadingComprehensionLearner

:

def

__init__

(

self

):

self

.

library

ArticleLibrary

()

self

.

generator

ReadingComprehensionGenerator

()

self

.

level_manager

ReadingLevelManager

()

self

.

current_article

None

self

.

current_questions

None

def

get_recommended_article

(

self

):

current_level

self

.

level_manager

.

get_current_level

()

articles

self

.

library

.

get_articles_by_difficulty

(

current_level

[

"level"

```
],  
1  
)  
if  
articles  
:  
self  
.  
current_article  
  
articles  
[  
0  
]  
summary  
  
self  
.  
generator  
.  
generate_summary  
(  
self  
.  
current_article  
[  
"content"
```

```
]
)
return
{
  "article"
:
  self
.
  current_article
,
  "summary"
:
  summary
,
  "current_level"
:
  current_level
}
else
:
return
{
  "error"
:
  "该级别暂无文章"
}
```

```
def
start_quiz
(
self
,
question_count

5
):
if
not
self
.
current_article
:
return
{
"error"
:
"请先获取文章"
}
questions

self
.
generator
.
```

```
generate_questions
```

```
(
```

```
self
```

```
.
```

```
current_article
```

```
[
```

```
"title"
```

```
]
```

```
,
```

```
self
```

```
.
```

```
current_article
```

```
[
```

```
"content"
```

```
]
```

```
,
```

```
question_count
```

```
)
```

```
self
```

```
.
```

```
current_questions
```

```
questions
```

```
return
```

```
questions
```

```
def
```

```
submit_answers
```

```
(  
    self  
    ,  
    answers  
):  
    if  
        not  
            self  
                .  
                    current_questions  
:  
    return  
    {  
        "error"  
        :  
        "请先开始测验"  
    }  
    correct_count  
  
    0  
    results  
  
    []  
    for  
        i  
        ,  
        question
```

```
in
enumerate
(
self
.
current_questions
[
"questions"
]
):
user_answer

answers
.
get
(
str
(
i
),
""
)
is_correct

user_answer
.
strip
```

()

.

upper

()

question

[

"answer"

]

.

strip

()

.

upper

()

if

is_correct

:

correct_count

+=

1

results

.

append

{

"question"

:

question

[

"question"

]

,

"your_answer"

:

user_answer

,

"correct_answer"

:

question

[

"answer"

]

,

"correct"

:

is_correct

,

"explanation"

:

question

[

"explanation"

]

```
    })

    total_count

    len

    (

    self

    .

    current_questions

    [

    "questions"

    ]

    )

    self

    .

    level_manager

    .

    record_score

    (

    correct_count

    ,

    total_count

    )

    return

    {

    "correct_count"

    :

    correct_count
```

```
,  
"total_count"  
:  
total_count  
,  
"accuracy"  
:  
round  
(  
correct_count  
/  
total_count  
,  
2  
)  
,  
"results"  
:  
results  
,  
"level_change"  
:  
self  
.  
level_manager  
.
```

```
get_current_level
```

```
()
```

```
}
```

常见问题

1. 文章难度评估不准确：可以调整DifficultyEvaluator中的CEFR映射规则
2. 题目生成质量不定：优化prompt或使用更强大的模型
3. 自适应调整过于频繁：增加_adjust_level中的样本数量要求
4. 文章库为空：使用ArticleLibrary.add_article添加示例文章

—— 未完待续 ——

系列导航：

- 本地大模型知识库系列（12篇，已完成）
- 英语学习Agent系列（8篇，当前第04篇）

英语学习Agent系列第04篇

下一篇：口语练习模块——对话式口语训练

5. 口语练习模块

口语练习模块——对话式口语训练

系列导航：

【本地大模型知识库系列】 01-12篇（已完成） |

【英语学习Agent系列】 当前第05篇（共8篇）

口语练习模块

对话式口语训练

英语学习Agent系列第05篇 | 口语练习

口语是英语学习中最具挑战性的部分之一。缺乏对话伙伴、害怕犯错、不知道如何改进——这些都是常见的痛点。我们将构建一个对话式口语训练系统，让学习者可以随时随地进行英语对话练习，并获得即时反馈和纠正。

01

对话场景设计

定义各种日常对话场景，让学习者在不同情境下练习口语：

```
python
from
enum
import
Enum
class
ConversationScenario
(
Enum
):
```

GREETING

"问候"

ORDERING_FOOD

"点餐"

SHOPPING

"购物"

TRAVEL

"旅行"

JOB_INTERVIEW

"面试"

DISCUSSION

"讨论"

APPOINTMENT

"预约"

COMPLAINT

"投诉"

class

ScenarioManager

:

def

__init__

(

self

):

self

.

scenarios

{

"GREETING"

:

{

"description"

:

"日常问候和自我介绍"

,

"level"

:

"A1"

,

"opening"

:

"Hi! How are you today?"

,

"target_expressions"

:

[

"Nice to meet you"

,

"How are you"

,

"I'm fine, thank you"

]

},

"ORDERING_FOOD"

:

{

"description"

:

"在餐厅点餐"

,

"level"

:

"A2"

,

"opening"

"Can I take your order?"

,

"target_expressions"

:

[

"I'd like"

,

"Could I have"

,

"With fries"

]

},

"SHOPPING"

:

{

"description"

:

"购物场景对话"

,

"level"

:

"B1"

,

"opening"

:

"Welcome to our store! Can I help you?"

,

"target_expressions"

:

[

"How much is this"

,

"Do you have"

,

"Can I try it on"

]

},

"TRAVEL"

:

{

"description"

:

"旅行相关对话"

,

"level"

:

"B1"

,

"opening"

:

"Good morning! Where are you heading today?"

,

"target_expressions"

:

[

"I'd like a ticket"

,

"How long does it take"

,

"Where is"

]

```
},  
  
"JOB_INTERVIEW"  
  
:  
  
{  
  
"description"  
  
:  
  
"求职面试"  
  
,  
  
"level"  
  
:  
  
"C1"  
  
,  
  
"opening"  
  
:  
  
"Good afternoon. Thank you for coming today. Please tell me about yourself."  
  
,  
  
"target_expressions"  
  
:  
  
[  
  
"I have experience in"  
  
,  
  
"My strengths are"  
  
,  
  
"I am proficient in"  
  
]  
  
}
```

```
}  
  
def  
get_scenario  
(  
self  
,  
name  
):  
return  
self  
.  
scenarios  
.  
get  
(  
name  
.upper  
( ),  
None  
)  
  
def  
list_scenarios  
(  
self  
):  
return
```

```
[{  
  
  "name"  
  
  :  
  
  k  
  
  ,  
  
  "description"  
  
  :  
  
  v  
  
  [  
  
    "description"  
  
  ]  
  
  ,  
  
  "level"  
  
  :  
  
  v  
  
  [  
  
    "level"  
  
  ]  
  
}  
  
for  
  
k  
  
,  
  
v  
  
in  
  
self  
  
.
```

scenarios

.

items

()

]

02

对话引擎实现

使用Ollama模拟对话伙伴，支持多轮对话：

python

import

ollama

import

json

class

ConversationEngine

:

def

__init__

(

self

,

model

"llama3:8b"

):

self

.

model

model

self

.

messages

[]

self

.

scenario

None

self

.

turn_count

0

def

start_conversation

(

self

,

scenario

):

self

.

scenario

scenario

self

.

messages

[]

self

.

turn_count

0

system_prompt

f"""

你是一位专业的英语对话练习伙伴。请在"{scenario['description']}"场景下与用户进行自然对话。

要求：

- 1. 使用{scenario['level']}级别的英语
- 2. 保持对话自然流畅，不要太机械
- 3. 根据用户的回答做出合适的回应
- 4. 不要主动结束对话，除非用户明确表示结束
- 5. 适当引导用户使用目标表达：{scenario['target_expressions']}

"""

self

.

messages

.

append

({

"role"

:

"system"

,

"content"

:

system_prompt

})

self

.

messages

.

append

{

"role"

:

"assistant"

,

"content"

:

scenario

[

"opening"

]

})

return

```
scenario

[
    "opening"
]

def
    respond
    (
        self
        ,
        user_input
    ):
        self
        .
        messages
        .
        append
        ({
            "role"
            :
            "user"
            ,
            "content"
            :
            user_input
        })
        response
```

ollama

.

chat

(

model

self

.

model

,

messages

self

.

messages

)

reply

response

[

"message"

][

"content"

]

self

.

messages

.

```
append

({

  "role"

  :

  "assistant"

  ,

  "content"

  :

  reply

})

self

.

turn_count

+=

1

return

reply

def

end_conversation

(

  self

):

return

{

  "scenario"

  :


```

```
self
.
scenario
[
"description"
]
,
"turn_count"
:
self
.
turn_count
,
"total_messages"
:
len
(
self
.
messages
)
}
```

03

口语评估与反馈

对用户的口语表达进行评估，提供即时反馈和改进建议：

python

```
import
ollama

import
json

class
SpeakingEvaluator

:

def
__init__
(
self
,
model

"llama3:8b"
):
self
.
model

model

def
evaluate_response
(
self
,
user_input
```

,

scenario

,

target_expressions

):

prompt

f"""

请评估用户在"{scenario}"场景下的英语口语表达：

用户输入：{user_input}

需要评估的维度：

- 1. 语法准确性（0–10分）
- 2. 词汇丰富度（0–10分）
- 3. 表达自然度（0–10分）
- 4. 目标表达使用（是否使用了这些表达：{target_expressions}）

请提供：

- 1. 综合评分（0–100分）
- 2. 具体的语法错误和纠正建议
- 3. 更地道的表达方式
- 4. 鼓励性反馈

请以JSON格式输出：

{{

"score": 综合评分,

"grammar_score": 语法得分,

"vocabulary_score": 词汇得分,

"fluency_score": 流畅度得分,

"grammar_errors": [{"error": "错误内容", "correction": "纠正后", "explanation": "解释"}],

```
"improvement_suggestions": ["建议1", "建议2", "建议3"],

"better_expressions": ["更好的表达1", "更好的表达2"],

"encouragement": "鼓励性话语"

}}
```

"""

response

ollama

.

chat

(

model

self

.

model

,

messages

"role"

:

"user"

,

"content"

:

prompt

}}

)

```
try
:
return
json
.
loads
(
response
[
"message"
][
"content"
]
)
except
json
.
JSONDecodeError
:
return
{
"score"
:
0
,
"error"
```

```
:  
  
"评估失败，请重试"  
  
}  
  
def  
  
generate_summary  
  
(  
  
self  
  
,  
  
conversation_history  
  
,  
  
evaluations  
  
):  
  
history_text  
  
"  
"  
  
.  
  
join  
  
(  
  
[  
  
f"{m['role']}: {m['content']}"  
  
for  
  
m  
  
in  
  
conversation_history  
  
if  
  
m  
  
[
```

"role"

] !=

"system"

]

)

prompt

f"""

请为以下对话生成口语练习总结：

对话内容：

{history_text}

评估记录：

{json.dumps(evaluations, ensure_ascii=False)}

请以JSON格式输出：

{{

"overall_score": 平均分,

"strengths": ["优点1", "优点2"],

"weaknesses": ["不足1", "不足2"],

"key_improvements": ["重点改进1", "重点改进2"],

"practice_suggestions": ["练习建议1", "练习建议2", "练习建议3"],

"new_vocabulary": ["新单词1", "新单词2", "新单词3"]

}}

"""

response

ollama

.

```
chat
(
model

self
.
model
,
messages

"role"
:
"user"
,
"content"
:
prompt
}]
)

try
:
return

json
.
loads
(
response
```

```
[  
    "message"  
]  
["  
    "content"  
]  
)  
except  
    json  
    .  
    JSONDecodeError  
:  
    return  
    {  
        "error"  
:  
        "总结生成失败"  
    }
```

04

口语学习流程

python

class

SpeakingPractice

:

def

__init__

(

self

):

self

.

scenario_manager

ScenarioManager

()

self

.

conversation_engine

ConversationEngine

()

self

.

evaluator

SpeakingEvaluator

()

self

.

evaluations

[]

self

.

is_active

```
False

def
start_practice
(
self
,
scenario_name
):
scenario

self
.
scenario_manager
.
get_scenario
(
scenario_name
)
if
not
scenario
:
return
{
"error"
:
"场景不存在"
}
```

```
}

opening_line

self

.

conversation_engine

.

start_conversation

(

scenario

)

self

.

is_active

True

self

.

evaluations

[]

return

{

"status"

:

"started"

,

"scenario"
```

```
:  
  
scenario  
  
[  
  
"description"  
  
]  
  
,  
  
"opening"  
  
:  
  
opening_line  
  
,  
  
"target_expressions"  
  
:  
  
scenario  
  
[  
  
"target_expressions"  
  
]  
  
}  
  
def  
  
speak  
  
(  
  
self  
  
,  
  
user_input  
  
):  
  
if  
  
not
```

```
self
.
is_active
:
return
{
"error"
:
"请先开始练习"
}
```

```
reply
```

```
self
.
conversation_engine
.
respond
(
user_input
)
evaluation
```

```
self
.
evaluator
.
evaluate_response
```

```
(  
    user_input  
  
    ,  
  
    self  
  
    .  
  
    conversation_engine  
  
    .  
  
    scenario  
  
    [  
        "description"  
    ]  
  
    ,  
  
    self  
  
    .  
  
    conversation_engine  
  
    .  
  
    scenario  
  
    [  
        "target_expressions"  
    ]  
    )  
  
    self  
  
    .  
  
    evaluations  
  
    .  
  
    append
```

```
(  
  evaluation  
)  
  
return  
  
{  
  "reply"  
  :  
  reply  
  ,  
  "evaluation"  
  :  
  evaluation  
  ,  
  "turn"  
  :  
  self  
  .  
  conversation_engine  
  .  
  turn_count  
}  
  
def  
  end_practice  
  (  
    self  
  ):  
  
```

```
if
not
self
.
is_active
:
return
{
"error"
:
"没有正在进行的练习"
}
summary

self
.
evaluator
.
generate_summary
(
self
.
conversation_engine
.
messages
,
self
```

.

evaluations

)

conversation_info

self

.

conversation_engine

.

end_conversation

()

self

.

is_active

False

return

{

"status"

:

"ended"

,

"conversation_info"

:

conversation_info

,

"summary"

```
:
summary
}

def
list_scenarios
(
self
):
return
self
.
scenario_manager
.
list_scenarios
()
```

常见问题

1. 对话不自然：优化system prompt，让模型更像真人对话
2. 评估不准确：增加评估维度或使用更强大的模型
3. 对话过长导致回复变慢：限制对话轮数或启用流式输出（参考【本地大模型知识库系列】第08篇）
4. 缺少语音识别：可以集成Whisper实现语音输入（参考【本地大模型知识库系列】第11篇）

—— 未完待续 ——

系列导航：

- 本地大模型知识库系列（12篇，已完成）
- 英语学习Agent系列（8篇，当前第05篇）

英语学习Agent系列第05篇

下一篇：写作辅助模块——AI写作教练

6. 写作辅助模块

写作辅助模块——AI写作教练

系列导航：

【本地大模型知识库系列】 01-12篇（已完成） |

【英语学习Agent系列】 当前第06篇（共8篇）

写作辅助模块

AI写作教练

英语学习Agent系列第06篇 | 写作辅助

写作是英语学习中最考验综合能力的部分。从构思到表达，从语法到用词，每一步都需要精心打磨。我们将构建一个AI写作教练，帮助学习者完成从选题、大纲、草稿到润色的全流程写作指导。

01

写作主题生成与大纲

使用Ollama生成写作主题和大纲：

```
python
```

```
import
```

```
ollama
```

```
import
```

```
json
```

```
class
```

```
TopicGenerator
```

```
:
```

```
def
```

```
__init__
```

```
(
```

```
self
,
model

"llama3:8b"

):

self
.
model

model

def
generate_topics
(
self
,
category
,
level

"B1"
,
count

5

):

prompt

f"""
```

请为英语学习者生成{count}个{category}主题的写作题目，难度级别为{level}。

要求：

- 1. 题目要有明确的写作方向
- 2. 适合{level}级别的词汇和语法要求
- 3. 每个题目附带简短的写作建议

请以JSON格式输出：

```
{{
  "topics": [
    {{
      "title": "题目名称",
      "description": "题目描述",
      "suggestion": "写作建议"
    }}
  ]
}}
```

response

ollama

.

chat

(

model

self

.

model

```
,
messages

"role"

:

"user"

,

"content"

:

prompt

}]

)

try

:

return

json

.

loads

(

response

[

"message"

][

"content"

]

)

except
```

```
json
.
JSONDecodeError
:
return
None
def
generate_outline
(
self
,
topic
,
level

"B1"
):
prompt

f"""
请为写作题目"{topic}"生成一个详细的大纲，难度级别为{level}。

大纲结构：

1. 引言（介绍主题）

2. 主体段落（分点论述）

3. 结论（总结观点）

要求：

1. 每个部分要有明确的要点
```

- 2. 提供关键句型和词汇建议
- 3. 适合{level}级别的写作要求

请以JSON格式输出：

```
{{
  "topic": "{topic}",
  "outline": [
    {{
      "section": "章节名称",
      "points": ["要点1", "要点2", "要点3"],
      "key_sentences": ["句型1", "句型2"],
      "key_vocabulary": ["词汇1", "词汇2", "词汇3"]
    }}
  ],
  "tips": ["写作提示1", "写作提示2"]
}}
```

"""

response

ollama

.

chat

(

model

self

.

model

```
,
messages

"role"

:

"user"

,

"content"

:

prompt

}]

)

try

:

return

json

.

loads

(

response

[

"message"

][

"content"

]

)

except
```

```
json
.  
JSONDecodeError  
:  
return  
None
```

02

作文评估与润色

对用户的作文进行全面评估，并提供润色建议：

```
python  
import  
ollama  
import  
json  
class  
WritingEvaluator  
:  
    def  
    __init__  
    (  
        self  
        ,  
        model  
        "llama3:8b"  
    ):  
        self
```

.

model

model

def

evaluate_essay

(

self

,

essay

,

topic

):

topic_text

topic

if

topic

else

"未指定"

prompt

f"""

请评估以下英语作文：

题目：{topic_text}

作文内容：

{essay}

评估维度（每项0–20分）：

- 1. 内容完整性
- 2. 组织结构
- 3. 语法准确性
- 4. 词汇多样性
- 5. 表达流畅性

请提供：

- 1. 各维度得分和总分（0–100分）
- 2. 具体的语法错误和纠正
- 3. 词汇替换建议（提升表达丰富度）
- 4. 结构优化建议
- 5. 整篇作文的修改版本

请以JSON格式输出：

```
{{
  "total_score": 总分,
  "scores": {{
    "content": 内容得分,
    "structure": 结构得分,
    "grammar": 语法得分,
    "vocabulary": 词汇得分,
    "fluency": 流畅性得分
  }},
  "grammar_errors": [{{"original": "原句", "corrected": "修改后", "explanation": "解释"}}],
  "vocabulary_suggestions": [{{"original": "原词", "suggestion": "建议词", "context": "使用场景"}}],
  "structure_suggestions": ["建议1", "建议2"],
  "revised_essay": "修改后的作文",
}
```

"comments": "综合评价"

}}

""

response

ollama

.

chat

(

model

self

.

model

,

messages

"role"

:

"user"

,

"content"

:

prompt

}]

)

try

:

```
return
```

```
json
```

```
.
```

```
loads
```

```
(
```

```
response
```

```
[
```

```
"message"
```

```
][
```

```
"content"
```

```
]
```

```
)
```

```
except
```

```
json
```

```
.
```

```
JSONDecodeError
```

```
:
```

```
return
```

```
{
```

```
"error"
```

```
:
```

```
"评估失败，请重试"
```

```
}
```

```
def
```

```
get_paragraph_feedback
```

```
(
```

self

,

paragraph

):

prompt

f"""

请对以下段落进行分析，并提供改进建议：

段落内容：

{paragraph}

请分析：

- 1. 语法问题
- 2. 用词是否恰当
- 3. 句子结构是否清晰
- 4. 连贯性如何

请提供具体的修改建议和优化后的版本。

请以JSON格式输出：

{{

"issues": ["问题1", "问题2"],

"suggestions": ["建议1", "建议2"],

"improved_version": "优化后的段落",

"explanation": "修改说明"

}}

"""

response

ollama

```
.
chat
(
model

self

.
model

,
messages

"role"

:

"user"

,

"content"

:

prompt
}]

)

try

:

return

json

.

loads

(
```

```
response
[
    "message"
    [
        "content"
    ]
]
)
except
    json
    .
    JSONDecodeError
    :
    return
    {
        "error"
        :
        "分析失败，请重试"
    }
```

03

写作辅助流程

```
python
class
```

```
WritingCoach
:
def
__init__
```

```
(  
    self  
):  
    self  
.  
topic_generator
```

TopicGenerator

```
()  
self  
.  
evaluator
```

WritingEvaluator

```
()  
self  
.  
current_topic
```

None

```
self  
.  
current_outline
```

None

```
def  
suggest_topics
```

```
(
```

```
self
,
category
,
level

"B1"
):
return
self
.
topic_generator
.
generate_topics
(
category
,
level
)
def
get_outline
(
self
,
topic
,
level
```

"B1"

):

outline

self

.

topic_generator

.

generate_outline

(

topic

,

level

)

if

outline

:

self

.

current_topic

topic

self

.

current_outline

outline

return

outline

def

evaluate_essay

(

self

,

essay

):

return

self

.

evaluator

.

evaluate_essay

(

essay

,

self

.

current_topic

)

def

get_paragraph_feedback

(

self

,

```
paragraph

):

return

self

.

evaluator

.

get_paragraph_feedback

(

paragraph

)

def

improve_sentence

(

self

,

sentence

):

prompt

f"""
```

请优化以下英语句子，使其更地道、更流畅：

原句：{sentence}

请提供：

- 1. 优化后的句子（保持原意）
- 2. 优化说明

请以JSON格式输出：

```
{{  
  "original": "{sentence}",  
  "improved": "优化后的句子",  
  "explanation": "优化说明"  
}}
```

```
"""
```

```
response
```

```
ollama
```

```
.
```

```
chat
```

```
(
```

```
model
```

```
self
```

```
.
```

```
evaluator
```

```
.
```

```
model
```

```
,
```

```
messages
```

```
"role"
```

```
:
```

```
"user"
```

```
,
```

```
"content"
```

```
:
```

prompt

}]

)

try

:

return

json

.

loads

(

response

[

"message"

][

"content"

]

)

except

json

.

JSONDecodeError

:

return

{

"error"

:

"优化失败，请重试"

}

常见问题

1. 生成的题目太难或太简单：调整level参数或优化prompt中的难度描述
2. 作文评估不准确：增加评估维度或使用更强大的模型
3. 修改版本偏离原意：在prompt中强调"保持原意不变"
4. 长文评估速度慢：限制作文长度或启用流式输出（参考【本地大模型知识库系列】第08篇）

—— 未完待续 ——

系列导航：

- 本地大模型知识库系列（12篇，已完成）
- 英语学习Agent系列（8篇，当前第06篇）

英语学习Agent系列第06篇

下一篇：Web界面集成——统一学习平台

7. Web界面集成

Web界面集成——统一学习平台

系列导航：

【本地大模型知识库系列】 01-12篇（已完成） |

【英语学习Agent系列】 当前第07篇（共8篇）

Web界面集成

统一学习平台

英语学习Agent系列第07篇 | Web界面

之前我们已经完成了词汇、语法、阅读、口语、写作五个核心模块的开发。现在需要将这些模块整合到一个统一的Web平台上，让用户可以一站式使用所有功能。我们将使用Gradio来构建这个界面（关于Gradio的使用，请参考【本地大模型知识库系列】 第04篇）。

01

平台架构设计

设计统一平台的模块结构和界面布局：

02

Gradio界面实现

使用Gradio构建统一学习平台的主界面：

```
python
import
gradio
as
gr
import
json
```

```
class
EnglishLearningPlatform

:

def
__init__
(
self
):
self
.
vocabulary_reviewer

None
self
.
grammar_learner

None
self
.
reading_learner

None
self
.
speaking_practice

None
```

self

.

writing_coach

None

self

.

_init_learners

()

def

_init_learners

(

self

):

try

:

from

vocabulary_reviewer

import

VocabularyReviewer

self

.

vocabulary_reviewer

VocabularyReviewer

()

except

ImportError

:

print

(

"Vocabulary module not available"

)

try

:

from

grammar_learner

import

GrammarLearner

self

.

grammar_learner

GrammarLearner

()

except

ImportError

:

print

(

"Grammar module not available"

)

try

:

```
from
reading_learner

import
ReadingComprehensionLearner

self

.

reading_learner

ReadingComprehensionLearner

()

except
ImportError

:

print

(
"Reading module not available"
)

try

:

from

speaking_practice

import

SpeakingPractice

self

.

speaking_practice
```

SpeakingPractice

()

except

ImportError

:

print

(

"Speaking module not available"

)

try

:

from

writing_coach

import

WritingCoach

self

.

writing_coach

WritingCoach

()

except

ImportError

:

print

(

"Writing module not available"

```
)  
  
def  
  
build_interface  
  
(  
  
self  
  
):  
  
with  
  
gr.Blocks  
  
(  
  
title  
  
  
"英语学习Agent"  
  
,  
  
theme  
  
  
  
gr  
  
.br/>  
themes  
  
.br/>  
Soft  
  
())  
  
as  
  
demo  
  
:  
  
gr  
  
.br/>  
Markdown
```

```
(  
    """  
  
    # 英语学习Agent  
  
    一个基于本地大模型的智能英语学习平台，支持词汇、语法、阅读、口语、写作全方位训练。  
  
    """  
  
)  
  
with  
  
gr.Tabs  
  
():  
  
    with  
  
gr.TabItem  
  
(  
    " 词汇学习"  
  
):  
  
    self  
  
    .  
  
    _build_vocabulary_tab  
  
()  
  
    with  
  
gr.TabItem  
  
(  
    " 语法学习"  
  
):  
  
    self  
  
    .  
  
    _build_grammar_tab
```

```
()

with

gr.TabItem

(

" 阅读理解"

):

self

.

_build_reading_tab

()

with

gr.TabItem

(

" 口语练习"

):

self

.

_build_speaking_tab

()

with

gr.TabItem

(

" 写作辅助"

):

self

.
```

```
_build_writing_tab

()

return

demo

python

def

_build_vocabulary_tab

(

self

):

if

not

self

.

vocabulary_reviewer

:

gr

.

Markdown

(

"词汇模块未加载"

)

return

word_input

gr

.
```

Textbox

(

label

"输入单词"

)

add_btn

gr

.

Button

(

"添加单词"

)

result_output

gr

.

JSON

(

label

"单词卡"

)

start_review_btn

gr

.

Button

```
(  
    "开始复习"  
)
```

```
review_output
```

```
gr
```

```
.
```

```
JSON
```

```
(
```

```
label
```

```
"复习内容"
```

```
)
```

```
add_btn
```

```
.
```

```
click
```

```
(
```

```
fn
```

```
self
```

```
.
```

```
vocabulary_reviewer
```

```
.
```

```
add_new_word
```

```
,
```

```
inputs
```

```
word_input
```

```
],  
  
outputs  
  
result_output  
  
]  
  
)  
  
start_review_btn  
  
.click  
  
(  
  
fn  
  
self  
  
.vocabulary_reviewer  
  
.start_review  
  
,  
  
outputs  
  
review_output  
  
]  
  
)  
  
def  
  
_build_grammar_tab  
  
(  
  
self  
  
):
```

```
if
not
self
.
grammar_learner
:
gr
.
Markdown
(
"语法模块未加载"
)
return
topic_dropdown

gr
.
Dropdown
(
choices

"基础句型"

,

"名词"

,

"冠词"

,
```

"动词"

,

"一般现在时"

,

"一般过去时"

,

"情态动词"

],

label

"选择语法点"

)

start_btn

gr

.

Button

(

"开始学习"

)

explanation_output

gr

.

Markdown

(

label

"讲解"

)

exercises_output

gr

.

JSON

(

label

"练习题"

)

start_btn

.

click

(

fn

self

.

_format_grammar_result

,

inputs

topic_dropdown

],

outputs

explanation_output

```
,
exercises_output
]
)
def
_format_grammar_result
(
self
,
topic
):
result

self
.
grammar_learner
.
start_topic
(
topic
)
if
result
[
"status"
] ==
"error"
```

```
:  
  
return  
  
result  
  
[  
  
"message"  
  
],  
  
[]  
  
explanation  
  
result  
  
[  
  
"explanation"  
  
]  
  
md_text  
  
f"## {explanation.get('topic', '')}\n\n"  
  
md_text  
  
+=  
  
f"***定义** : {explanation.get('definition', '')}\n\n"  
  
md_text  
  
+=  
  
f"***规则** : \n"  
  
for  
  
rule  
  
in  
  
explanation  
  
.
```

```
get
(
"rules"
,
[]
):
md_text
+=
f"- {rule}\\n"
return
md_text
,
result
[
"exercises"
]
python
def
_build_reading_tab
(
self
):
if
not
self
.
```

reading_learner

:

gr

.

Markdown

(

"阅读模块未加载"

)

return

get_article_btn

gr

.

Button

(

"获取推荐文章"

)

article_output

gr

.

Markdown

(

label

"文章内容"

)

summary_output

```
gr
.
JSON
(
label

"摘要"

)
start_quiz_btn
```

```
gr
.
Button
(
"开始测验"

)
quiz_output
```

```
gr
.
JSON
(
label

"题目"

)
get_article_btn
.
```

```
click
(
fn

self
.
_format_article_result
,
outputs

article_output
,
summary_output
]
)

start_quiz_btn
.
click
(
fn

self
.
reading_learner
.
start_quiz
,
outputs
```

```
quiz_output

]

)

def

_format_article_result

(

self

):

result

self

.

reading_learner

.

get_recommended_article

()

if

"error"

in

result

:

return

result

[

"error"

],

[]
```

article

result

[

"article"

]

md_text

f"## {article.get('title', '')}\n\n"

md_text

+=

f"***难度** : {article.get('cefr_level', '')}\n\n"

md_text

+=

article

.

get

(

"content"

,

""

)

return

md_text

,

result

[

"summary"

]

python

def

_build_speaking_tab

(

self

):

if

not

self

.

speaking_practice

:

gr

.

Markdown

(

"口语模块未加载"

)

return

scenario_dropdown

gr

.

Dropdown

(

choices

"GREETING"

,

"ORDERING_FOOD"

,

"SHOPPING"

,

"TRAVEL"

,

"JOB_INTERVIEW"

],

label

"选择场景"

)

start_btn

gr

.

Button

(

"开始对话"

)

opening_output

gr

.

Markdown

(

label

"对话开始"

)

user_input

gr

.

Textbox

(

label

"你的回答"

)

send_btn

gr

.

Button

(

"发送"

)

reply_output

gr

.

Markdown

```
(
    label

    "回复"

)

evaluation_output

gr

.

JSON

(
    label

    "评估"

)

start_btn

.

click

(
    fn

    self

    .

    _format_speaking_start

    ,

    inputs

    scenario_dropdown

],
```

outputs

opening_output

]

)

send_btn

.

click

(

fn

self

.

_format_speaking_reply

,

inputs

user_input

],

outputs

reply_output

,

evaluation_output

]

)

def

_format_speaking_start

```
(  
  
self  
  
,  
  
scenario  
  
):  
  
result  
  
self  
  
.  
  
speaking_practice  
  
.  
  
start_practice  
  
(  
  
scenario  
  
)  
  
return  
  
f"## {result.get('scenario', '')}\n\n**AI**:{result.get('opening', '')}"  
  
def  
  
_format_speaking_reply  
  
(  
  
self  
  
,  
  
input_text  
  
):  
  
result  
  
self
```

.

speaking_practice

.

speak

(

input_text

)

return

f"***AI**:{result.get('reply', '')}"

,

result

[

"evaluation"

]

python

def

_build_writing_tab

(

self

):

if

not

self

.

writing_coach

:

```
gr
.
Markdown
(
"写作模块未加载"
)
return
category_input
```

```
gr
.
Textbox
(
label

"主题类别（如：教育、科技、生活）"
)
suggest_topics_btn
```

```
gr
.
Button
(
"推荐题目"
)
topics_output
```

```
gr
```

.

JSON

(

label

"题目列表"

)

topic_input

gr

.

Textbox

(

label

"输入写作题目"

)

get_outline_btn

gr

.

Button

(

"获取大纲"

)

outline_output

gr

.

JSON

(

label

"大纲"

)

essay_input

gr

.

Textbox

(

label

"输入作文"

,

lines

)

evaluate_btn

gr

.

Button

(

"评估作文"

)

evaluation_output

gr

.

JSON

(

label

"评估结果"

)

suggest_topics_btn

.

click

(

fn

self

.

writing_coach

.

suggest_topics

,

inputs

category_input

],

outputs

topics_output

]

)

get_outline_btn

.

click

(

fn

self

.

writing_coach

.

get_outline

,

inputs

topic_input

],

outputs

outline_output

]

)

evaluate_btn

.

click

(

fn

```
self
.
writing_coach
.
evaluate_essay
,
inputs

essay_input
],
outputs

evaluation_output
]
)

# 运行平台

if
__name__

"__main__"
:

platform

EnglishLearningPlatform
()

demo

platform
.
```

```
build_interface
```

```
()
```

```
demo
```

```
.
```

```
launch
```

```
(
```

```
server_name
```

```
"0.0.0.0"
```

```
,
```

```
server_port
```

```
7860
```

```
)
```

常见问题

1. 模块导入失败：确保所有模块文件在同一目录下
2. Gradio界面样式问题：尝试更换theme参数
3. 界面响应慢：优化Ollama调用速度（参考【本地大模型知识库系列】第10篇）
4. 端口被占用：修改server_port参数

—— 未完待续 ——

系列导航：

- 本地大模型知识库系列（12篇，已完成）
- 英语学习Agent系列（8篇，当前第07篇）

英语学习Agent系列第07篇

下一篇：项目部署与总结——打造你的专属英语学习助手

8. 项目部署与总结

项目部署与总结——打造你的专属英语学习助手

系列导航：

【本地大模型知识库系列】 01-12篇（已完成） |

【英语学习Agent系列】 当前第08篇（共8篇）

项目部署与总结

打造你的专属英语学习助手

英语学习Agent系列第08篇 | 部署总结

经过前面7篇文章的学习，我们已经完成了英语学习Agent的全部核心模块开发。现在，让我们将这些模块整合起来，进行项目部署，并对整个系列进行总结。

01

项目结构整理

首先，让我们整理项目结构，确保所有模块文件组织清晰：

bash

english-learning-agent/

|——

main.py

主入口，启动Gradio界面

|——

requirements.txt

依赖列表

|——

vocabulary_reviewer.py

词汇学习模块（第02篇）

|——

grammar_learner.py

语法学习模块（第03篇）

|——

reading_learner.py

阅读理解模块（第04篇）

|——

speaking_practice.py

口语练习模块（第05篇）

|——

writing_coach.py

写作辅助模块（第06篇）

|——

knowledge_graph.py

知识图谱（第03篇）

|——

data/

数据目录

| |——

vocabulary.db

单词数据库

| |——

grammar_topics.json

语法知识点

|——

chroma_db/

向量数据库（第04篇）

创建requirements.txt文件，列出所有必要的依赖：

bash

ollama

0.1.35

gradio

4.36.1

langchain

0.1.14

chromadb

0.4.22

networkx

3.2.1

textstat

0.7.3

sqlalchemy

2.0.29

python-dotenv

1.0.1

02

一键启动脚本

创建一键启动脚本，方便用户快速运行：

bash

#!/bin/bash

echo

" 启动英语学习Agent..."

if

! command

ollama

&>/dev/null

;

then

echo

" Ollama未安装，请先安装Ollama（参考【本地大模型知识库系列】第02篇） "

exit

1

fi

if

! ollama

list

2>/dev/null

;

then

echo

" 启动Ollama服务..."

ollama

serve

&

```
sleep
5
fi
if
! ollama
list
2>/dev/null
| grep -q
"llama3"
;
then
echo
" 下载llama3模型..."
ollama
pull
llama3
fi
echo
" 启动Web界面..."
python3
main.py
03
```

Docker容器化部署

对于需要在服务器上定运行的场景，可以使用Docker进行容器化部署。创建Dockerfile：

```
dockerfile
```

```
FROM
```

python:3.11-slim

WORKDIR

/app

RUN

apt-get update && apt-get install -y \

curl \

&& rm -rf /var/lib/apt/lists/*

RUN

curl -fsSL https://ollama.com/install.sh | sh

COPY

requirements.txt .

RUN

pip install --no-cache-dir -r requirements.txt

COPY

..

EXPOSE

7860

CMD

["bash", "-c", "ollama serve & sleep 5 && python main.py"]

创建docker-compose.yml文件：

yml

version

:

'3.8'

services

:

english-learning-agent

:

build

:

.

ports

:

"7860:7860"

volumes

:

./data:/app/data

./chroma_db:/app/chroma_db

~/.ollama:/root/.ollama

restart

:

"always"

04

性能优化

策略

为了确保学习Agent的流畅运行，我们可以采取以下优化策略（详细内容请参考【本地大模型知识库系列】第10篇）：

优化维度

优化策略

预期效果

模型选择

使用量化模型（如llama3:7b-q4_0）

内存占用降低50%

缓存机制

对高频查询结果进行缓存

响应速度提升30%

检索优化

限制检索文档数量，使用语义摘要

Token消耗减少40%

异步处理

使用Celery异步处理耗时任务

用户体验显著提升

05

系列总结

与展望

至此，英语学习Agent系列全部完成。让我们回顾一下整个系列的内容：

系列文章回顾

•

第01篇

：架构设计——为什么需要英语学习Agent

•

第02篇

：词汇学习——基于记忆曲线的智能单词本

•

第03篇

：语法学习——知识图谱驱动的学习路径

•

第04篇

：阅读理解——自适应难度的阅读训练

•

第05篇

：口语练习——对话式场景训练

•

第06篇

：写作辅助——AI驱动的作文评估

•

第07篇

：Web界面——统一学习平台

•

第08篇

：项目部署——打造专属学习助手

整个英语学习Agent系统基于【本地大模型知识库系列】搭建的基础设施，实现了从词汇到写作的全链路英语学习能力。核心技术包括：

Ollama本地大模型

：提供语言理解和生成能力（参考【本地大模型知识库系列】第02篇）

Chroma向量数据库

：支持文章检索和语义匹配（参考【本地大模型知识库系列】第03篇）

Gradio Web界面

：提供友好的用户交互体验（参考【本地大模型知识库系列】第04篇）

知识图谱

：构建语法知识点依赖关系，支持学习路径规划

间隔重复算法

：基于记忆曲线实现科学复习

在未来，你可以继续扩展这个系统：

听力训练模块

：集成TTS和语音识别，实现听写练习

多语言支持

：扩展到其他语言学习场景

移动端适配

：开发手机App或小程序

学习社区

：添加用户分享和排行榜功能

AI教师角色

：添加更多个性化的学习引导功能

常见问题

1. Docker部署失败：确保Docker版本支持GPU（如需）
2. 模型下载慢：配置Ollama镜像源（参考【本地大模型知识库系列】第02篇）
3. 内存不足：使用更小的量化模型，如q4_0或q2_k
4. 数据持久化：确保Docker volumes正确挂载

—— 系列完结 ——

系列导航：

- 本地大模型知识库系列（12篇，已完成）
- 英语学习Agent系列（8篇，全部完成）

英语学习Agent系列第08篇（完结篇）

感谢你的一路陪伴！

祝你英语学习进步！